A Major Project Report

On

REINFORCEMENT LEARNING FOR OPTIMAL ROBOTIC ARM CONTROL IN BALL-BALANCING TASKS FOR AUTOMATED MNUFACTURING

Submitted in the partial fulfilment of the requirements for the

Award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND INSTRUMENTATION ENGINEERING

By

SANDEEP VADLAMUDI	208W1A1054
AKHILA YARAMALA	208W1A1059
CHARUMATHI UPPALAPATI	208W1A1052
PAVAN KUMAR KOLLURI	198W1A1026

Under the guidance of

KORUPU VIJAYA LAKSHMI, M. TECH (Ph. D)

DESIGNATION

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

V.R. SIDDHARTHA ENGINEERING COLLEGE (AUTONOMOUS)

(Affiliated to JNTUK, Kakinada)

Sponsored by SAGTE, Kanuru, Vijayawada - 520 007

(Approved by AICTE, Accredited by NBA and NAAC A+ GRADE)

2023 - 24

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING

V.R. SIDDHARTHA ENGINEERING COLLEGE (Autonomous)

CERTIFICATE

This is to certify that the Major Project report titled "**REINFORCEMENT LEARNING FOR OPTIMAL ROBOTIC ARM CONTROL IN BALL-BALANCING TASKS FOR AUTOMATED MANUFACTURING**" is a bona fide record of work done by **SANDEEP VADLAMUDI (208W1A1054), AKHILA YARAMALA (208W1A1059), CHARUMATHI UPPALAPATI (208W1A1052), PAVAN KUMAR KOLLURI (198W1A1026)** under my guidance and supervision and is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics & Instrumentation Engineering, V.R. Siddhartha Engineering College, (Autonomous, Affiliated to JNTUK, Kakinada) during the academic year 2023-2024.

KORUPU VIJAYA LAKSHMI
M. TECH (Ph. D)
Asst. Professor,
Dept. of EIE.

Dr. G.N. SWAMY
M.Tech., Ph.D.,
Professor & Head,
Dept. of EIE.

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our guide **KORUPU VIJAYA LAKSHMI, Asst. Professor, Department of Electronics and Instrumentation Engineering, V.R. Siddhartha Engineering College, Kanuru, Vijayawada.** S/he has given us tremendous support in both technical and moral front. Without his/her support and encouragement, we would never have been able to complete the Major Project successfully.

We are profoundly grateful to **Dr. G.N. Swamy, Professor, Head of the department, Electronics and Instrumentation Engineering, V.R. Siddhartha Engineering College, Kanuru, Vijayawada,** for his kind and whole hearted support and valuable guidance throughout the Major Project.

We are also grateful to Project Review Committee members for their supervision and valuable suggestions and other teaching and non-teaching staff for their kind co-operation.

We are thankful **to Dr. A.V. Ratnaprasad, Principal, V.R. Siddhartha Engineering College Kanuru, Vijayawada,** for his support during the completion of the Major Project.

We would like to express our gratitude to our project coordinator, **Dr. Mahammad Firose Shaik, Assistant Professor,** for his utmost support and encouragement and regular monitoring of our work throughout the Major Project.

We would also like to express our special gratitude and thanks to **Dr. P. Srinivas, Associate Professor** (and any other person) for providing the information to us.

V. SANDEEP
Y. AKHILA
U. CHARUMATHI
K. PAVAN KUMAR

.

INDEX

LIST OF FIGURES

1 INTRODUCTION

A ball-and-plate platform is a common example of the dynamical system used to apply modelling and control concepts studied in some engineering courses. In this kind of systems, a spherical body is positioned on a mobile plate with two degrees of freedom. The system is operated by two servo motors that cause displacement of the sphere in the x and y axis in the plane of the platform. It is known that such a system is highly unstable and non-linear. Initially, from a phenomenological analysis we obtained a state-space representation of the system. Then, a state feedback controller using the pole assignment method was designed to maintain a desired position of the sphere over the plane. A computer vision technique allowed us measuring the two-dimensional position of the sphere over the plane in real time. Model of the plant under study and controller design for sphere positioning were experimentally validated considering different set-points. Obtained results let us know that our plant controller is stable and responds satisfactorily to position disturbances around a reference point.

Our approach to solve the proposed problem is using state of the art reinforcement learning algorithm in Robot Operating System (ROS) environment. We will be using reinforcement learning (RL) to overcome challenge of data gathering, because reinforcement learning does not need any data for solving a problem instead it learns by reward and punishment technique. There are many reinforcement learning algorithms, but we will be using Deep Quality Network (DQN) which is recently attract many applications and emerged in recent research for variety type of application where human type of decision is needed. We will be using ROS in combination with Gazebo simulation which is highly realistic environment and highly used in commercial and research problems. Using the realistic environment helps on computation time and overcome challenge of safety. Safety is a key problem in reinforcement learning because agent or robot does not have any understanding of its environment at the beginning and needs to explore and find right action in right situation over time. So, it may break or do very unsafe actions during learning process. We will also make the learning model very robust by applying forces randomly during training process. So, the robust model can be used in real robot and adopt itself with the new dynamic model. Our proposed solution overcome all challenges that mentioned for this specific problem. We will be using a sensor on the top of the inverted pendulum to sense the angular velocity and orientation of the inverted pendulum, these data used as input to the learning algorithm. We use sensor fusion technique to make this sensor and it is explained in detail on hardware configuration. Since majority of recent publications are single degree freedom inverted

pendulum, to better analyzing and proper apple to apple comparison with their results, we have broken down the complex problem into 3 phases, from simple system which is one degree freedom inverted pendulum to more complex system, which is 3 DoF inverted pendulum on a 6 degrees robot arm.

2 REVIEW OF LITERATURE

Our soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to enable stability and exploration. We review prior works that draw on some of these ideas in this section. Actor-critic algorithms are typically derived starting from policy iteration, which alternates between policy evaluation computing the value function for a policy and policy improvement using the value function to obtain a better policy (Barto et al., 1983; Sutton & Barto, 1998). In large-scale reinforcement learning problems, it is typically impractical to run either of these steps to convergence, and instead the value function and policy are optimized jointly. In this case, the policy is referred to as the actor, and the value function as the critic. Many actor-critic algorithms build on the standard, on-policy policy gradient formulation to update the actor (Peters & Schaal, 2008), and many of them also consider the entropy of the policy, but instead of maximizing the entropy, they use it as an regularizer (Schul-man et al., 2017b; 2015; Mnih et al., 2016; Gruslys et al., 2017). On-policy training tends to improve stability but results in poor sample complexity.

There have been efforts to increase the sample efficiency while retaining robustness by incorporating off-policy samples and by using higher order variance reduction techniques (O'Donoghue et al., 2016; Gu et al., 2016). How-ever, fully off-policy algorithms still attain better efficiency. A particularly popular off-policy actor-critic method, DDPG (Lillicrap et al., 2015), which is a deep variant of the deterministic policy gradient (Silver et al., 2014) algorithm, uses a Q-function estimator to enable off-policy learning, and a deterministic actor that maximizes this Q-function. As such, this method can be viewed both as a deterministic actor-critic algorithm and an approximate Q-learning algorithm. Unfortunately, the interplay between the deterministic actor network and the Q-function typically makes DDPG extremely difficult to stabilize and brittle to hyperparameter settings (Duan et al., 2016; Henderson et al., 2017). As a consequence, it is difficult to extend DDPG to complex, high-dimensional tasks, and on-policy policy gradient methods still tend to produce the best results in such settings

(Guet al., 2016). Our method instead combines off-policy actor-critic training with a stochastic actor, and further aims to maximize the entropy of this actor with an entropy maximization objective. We find that this actually results in a considerably more stable and scalable algorithm that, in practice, exceeds both the efficiency and final performance of DDPG. A similar method can be derived as a zero-step special case of stochastic value gradients (SVG) (Heesset al., 2015). However, SVG differs from our method in that it optimizes the standard maximum expected return objective, and it does not make use of a separate value network, which we found to make training more stable.

Maximum entropy reinforcement learning optimizes policies to maximize both the expected return and the expected entropy of the policy. This framework has been used in many contexts, from inverse reinforcement learning (Ziebart et al., 2008) to optimal control (Todorov, 2008; Toussaint, 2009; Rawlik et al., 2012). In guided policy search (Levine & Koltun, 2013; Levine et al., 2016), the maximum entropy distribution is used to guide policy learning towards high-reward regions. More recently, several papers have noted the connection between Q-learning and policy gradient methods in the framework of maximum entropy learning (O'Donoghue et al., 2016; Haarnoja et al., 2017; Nachum et al., 2017a; Schulman et al., 2017a). While most of the prior model-free works assume a discrete action space, Nachum et al. (2017b) approximate the maximum entropy distribution with a Gaussian and Haarnoja et al. (2017) with a sampling network trained to draw samples from the optimal policy. Although the soft Q-learning algorithm pro-posed by Haarnoja et al. (2017) has a value function and actor network, it is not a true actor-critic algorithm: the Q-function is estimating the optimal Q-function, and the actor does not directly affect the Q-function except through the data distribution. Hence, Haarnoja et al. (2017) motivates the actor network as an approximate sampler, rather than the actor in an actor-critic algorithm. Crucially, the convergence of this method hinges on how well this sampler approximates the true posterior. In contrast, we prove that our method converges to the optimal policy from a given policy class, regardless of the policy parameterization. Furthermore, these prior maximum entropy methods generally do not exceed the performance of state-of-the-art off-policy algorithms, such as DDPG, when learning from scratch, though they may have other benefits, such as improved exploration and ease of fine-tuning. In our experiments, we demonstrate that our soft actor-critic algorithm does in fact exceed the performance of prior state-of-the-art off-policy deep RL methods by a wide margin.

3 MATLAB SOFTWARE

MATLAB (an abbreviation of "MATrix LABoratory") is a proprietary multi-paradigm programming language and numeric computing environment developed by MathWorks. MATLAB allows matrix manipulations, plotting of functions and data, implementation of algorithms, creation of user interfaces, and interfacing with programs written in other languages. Although MATLAB is intended primarily for numeric computing, an optional toolbox uses the MuPAD symbolic engine allowing access to symbolic computing abilities. An additional package, Simulink, adds graphical multi-domain simulation and model-based design for dynamic and embedded systems.

As of 2020, MATLAB has more than four million users worldwide. They come from various backgrounds of engineering, science, and economics. As of 2017, more than 5000 global colleges and universities use MATLAB to support instruction and research. MATLAB was invented by mathematician and computer programmer Cleve Moler. The idea for MATLAB was based on his 1960s PhD thesis. Moler became a math professor at the University of New Mexico and started developing MATLAB for his students as a hobby. He developed MATLAB's initial linear algebra programming in 1967 with his one-time thesis advisor, George Forsythe. This was followed by Fortran code for linear equations in 1971.

Before version 1.0, MATLAB "was not a programming language; it was a simple interactive matrix calculator. There were no programs, no toolboxes, no graphics. And no ODEs or FFTs. The first early version of MATLAB was completed in the late 1970s. The software was disclosed to the public for the first time in February 1979 at the Naval Postgraduate School in California. Early versions of MATLAB were simple matrix calculators with 71 pre-built functions. At the time, MATLAB was distributed for free to universities. Moler would leave copies at universities he visited and the software developed a strong following in the math departments of university campuses.

In the 1980s, Cleve Moler met John N. Little. They decided to reprogram MATLAB in C and market it for the IBM desktops that were replacing mainframe computers at the time. John Little and programmer Steve Bangert re-programmed MATLAB in C, created the MATLAB programming language, and developed features for toolboxes. Since 1993 an open source alternative, GNU Octave (mostly compatible with matlab) and scilab (similar to matlab) have been available.

3.1 Features of MATLAB

MATLAB stands for Matrix laboratory. It was developed by Math works, and it is a multipurpose (or as we say it Multi-paradigm) programming language. It allows matrix manipulations and helps us to plot different types of functions and data. It can also be used for the analysis and design as such as the control systems. MATLAB is generally used for these types of tasks:

- Signal processing
- Optimization of functions
- Control system design
- Image and Audio processing
- Machine learning and Deep learning

MATLAB is a high-level language: MATLAB Supports Object oriented programming. It also supports different types of programming constructs like Control flow statements (IF-ELSE, FOR, WHILE). MATLAB also supports structures like in C programming, Functional programming (writing functions to contain commonly used code and later calling them). It also contains Input / Output statements like disp() and input().

Interactive graphics: MATLAB has inbuilt graphics to enhance user experience. We can actually visualize whatever data is there in forms of plots and figures. It also supports processing of image and displaying them in 2D or 3D formats. We can visualize and manipulate our data across any of the three dimensions (1D, 2D, and 3D). We can plot the functions and customize them also according to our needs like changing bullet points, line color and displaying/not displaying grid.

A large library of Mathematical functions: MATLAB has a huge inbuilt library of functions required for mathematical analysis of any data. It has common math functions like sqrt. factorial etc. It has functions required for statistical analysis like median, mode and std (to find standard deviation), and much more. MATLAB also has functions for signal processing like filter, butter (Butterworth filter design) audio read, Conv, xcorr, fft, fftshift etc. It also supports image processing and some common functions required for image processing in MATLAB are rgb2gray, rgb2hsv, adaptthresh etc.

Data access and processing: MATLAB allows accessing of data from external sources like image files (.jpg, .PNG), audio files (.mp), and real-time data from JDBC/ ODBC. We can

easily read data from external sources using the inbuilt MATLAB functions like audioread for reading audio files and imread for reading external images.

Interactive environment: MATLAB offers interactive environment by providing a GUI (Graphical user interface) and different types of tools like signal analyses and tuners. MATLAB also has tools for debugging and the development of any software. Importing and exporting files becomes easy in MATLAB through the GUI. We can view the workspace data as we progress in the development of our software and modify it according to our needs.

MATLAB can interface with different languages: We can write a set of codes (libraries) in languages like PERL and JAVA, and we can call those libraries from within the MATLAB itself. MATLAB also supports ActiveX and .NET libraries.

MATLAB and Simulink: MATLAB has an inbuilt feature of Simulink wherein we can model the control systems and see their real-time behavior. We can design any system either using code or building blocks and see their real-time working through various inbuilt tools. It has lucid examples of basic control systems and their working.

MATLAB's Application programming interface (API): MATLAB consists of an extensive API. Through this API, we can link our C/C++ programs directly to MATLAB. Some options available in MATLAB API are calling MATLAB programs, read and write M-files, and using MATLAB as an interface to run applications. MATLAB can be used both as a computation and analysis tool.

Machine Learning, Deep Learning, and Computer vision: The most demanding technologies like Machine learning, Deep learning, and Computer vision can be done in MATLAB. We can create and interconnect layers of a deep neural network, We can build custom training loops and training layers with automatic differentiation. For machine learning, we can use the DBSCAN algorithm to discover clusters and noise in DATA. For computer vision, we can do object tracking, object recognition, gesture recognition, and processing 3D point clouds.

Computational Biology toolbox: This toolbox provides a great way for biologists and researchers to create and analyze new algorithms and patterns for development in biological and biochemical domains. We can build biological models and analyze them using this toolbox. Moreover, for students, this toolbox can be very much educational if they want to explore the biological domain.

3.2 Advantages of MATLAB

- **Easy to use interface:** A user-friendly interface with features you want to use is one click away.
- **A large inbuilt database of algorithms:** MATLAB has numerous important algorithms you want to use already built-in, and you just have to call them in your code.
- **Extensive data visualization and processing:** We can process a large amount of data in MATLAB and visualize them using plots and figures.
- **Debugging of codes easy:** There are many inbuilt tools like analyzer and debugger for analysis and debugging of codes written in MATLAB**.**
- **Easy symbolic manipulation:** We can perform symbolic math operations in MATLAB using the symbolic manipulation algorithms and tools in MATLAB.

3.3 Disadvantages of MATLAB

- MATLAB is slow since it is an interpreted language that is MATLAB programs are not converted into Machine language but are run by external software, so it can sometimes be slow.
- We cannot create the OUTPUT file in MATLAB.
- One cannot use graphics in MATLAB with -nojvm option, on doing so, we will get a runtime error.
- We cannot make functions in one single .m file as we have in the case of other programming languages. We have to create different files for different functions.
- Sometimes, the error messages are not much informative, so you have to figure out the error yourself.

3.4 Applications of MATLAB

MATLAB can be used as a tool for simulating various electrical networks but the recent developments in MATLAB make it a very competitive tool for Artificial Intelligence, Robotics, Image processing, Wireless communication, Machine learning, Data analytics and whatnot. Though its mostly used by circuit branches and mechanical in the engineering domain to solve a basic set of problems its application is vast. It is a tool that enables computation, programming and graphically visualizing the results.

The basic data element of MATLAB as the name suggests is the Matrix or an array. MATLAB toolboxes are professionally built and enable you to turn your imaginations into reality.

MATLAB programming is quite similar to C programming and just requires a little brush up of your basic programming skills to start working with.

Below are a few applications of MATLAB

Statistics and machine learning (ML): This toolbox in MATLAB can be very handy for the programmers. Statistical methods such as descriptive or inferential can be easily implemented. So is the case with machine learning. Various models can be employed to solve modern-day problems. The algorithms used can also be used for big data applications.

Curve fitting: The curve fitting toolbox helps to analyze the pattern of occurrence of data. After a particular trend which can be a curve or surface is obtained, its future trends can be predicted. Further plotting, calculating integrals, derivatives, interpolation, etc can be done.

Control systems: Systems nature can be obtained. Factors such as closed-loop, open-loop, its controllability and observability, Bode plot, Nyquist plot, etc can be obtained. Various controlling techniques such as PD, PI and PID can be visualized. Analysis can be done in the time domain or frequency domain.

Signal Processing: Signals and systems and digital signal processing are taught in various engineering streams. But MATLAB provides the opportunity for proper visualization of this. Various transforms such as Laplace, Z, etc can be done on any given signal. Theorems can be validated. Analysis can be done in the time domain or frequency domain. There are multiple built-in functions that can be used.

Mapping: Mapping has multiple applications in various domains. For example, in big data, the MapReduce tool is quite important which has multiple applications in the real world. Theft analysis or financial fraud detection, regression models, contingency analysis, predicting techniques in social media, data monitoring, etc can be done by data mapping.

Deep learning: It's a subclass of machine learning which can be used for speech recognition, financial fraud detection, medical image analysis. Tools such as time-series, Artificial neural network (ANN), Fuzzy logic or combination of such tools can be employed.

Financial analysis: An entrepreneur before starting any endeavor needs to do a proper survey and the financial analysis in order to plan the course of action. The tools needed for this are all available in MATLAB. Elements such as profitability, solvency, liquidity, and stability can be identified. Business valuation, capital budgeting, cost of capital, etc can be evaluated.

Image processing: The most common application that we observe almost every day are bar code scanners, selfie (face beauty, blurring the background, face detection), image enhancement, etc. The digital image processing also plays quite an important role in transmitting data from far off satellites and receiving and decoding it in the same way. Algorithms to support all such applications are available.

Text analysis: Based on the text, sentiment analysis can be done. Google gives millions of search results for any text entered within a few milliseconds. All this is possible because of text analysis. Handwriting comparison in forensics can be done. No limit to the application and just one software which can do this all.

Electric vehicles designing: Used for modeling electric vehicles and analyze their performance with a change in system inputs. Speed torque comparison, designing and simulating of a vehicle, whatnot.

Aerospace: This toolbox in MATLAB is used for analyzing the navigation and to visualize flight simulator.

Audio toolbox: Provides tools for audio processing, speech analysis, and acoustic measurement. It also provides algorithms for audio and speech feature extraction and audio signal transformation.

3.5 MATLAB Simulink Tool Box

The Simulink library browser contains the collection of multiple tools and their functions. It is useful for the simulation of the dynamic system in the MATLAB environment.

The Simulink toolboxes provide the specific tools for analyzing, designing, simulation of the system, making the communication between the other system, etc.

And also, these toolboxes are required for building the system, designing automatic control systems, an image designing system, data designing system, algorithms, etc.

The MATLAB toolbox contains multiple functional tools as per your requirements for building dynamic systems or projects.

From Math works, the list of professional toolboxes is given below.

- Control System Toolbox
- DSP Communication Toolbox
- Fuzzy Logic Toolbox

- OPC Toolbox
- Data Acquisition Toolbox
- Image Acquisition Toolbox
- Robust Control Toolbox
- Vehicle Network Toolbox
- Neural Network Toolbox
- Instrument Control Toolbox
- System Identification Toolbox
- Communication System Toolbox
- Computer Vision System Toolbox

3.6 Flow Chart

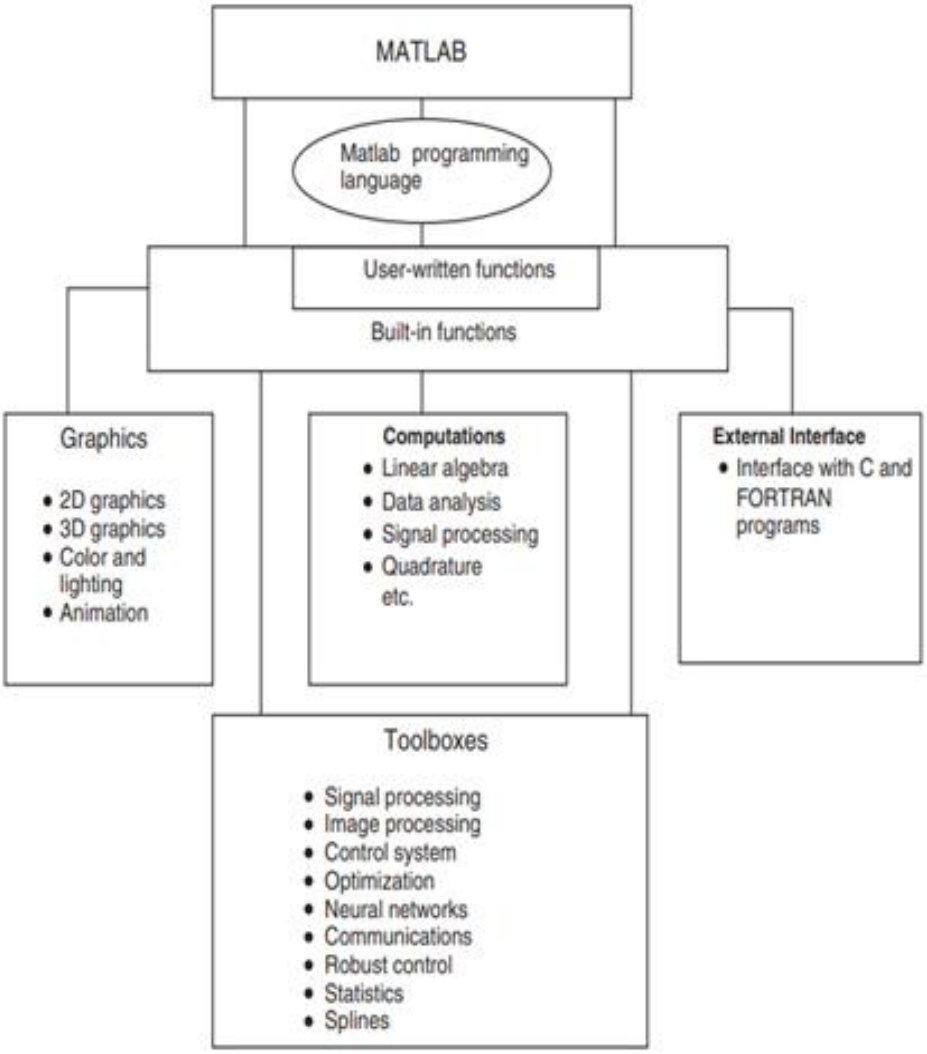


Fig.1. Features and capabilities of MATLAB

4 REINFORCEMENT LEARNING

Reinforcement learning is an area of Machine Learning. It is about taking suitable action to maximize reward in a particular situation. It is employed by various software and machines to find the best possible behavior or path it should take in a specific situation. Reinforcement learning differs from supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task. In the absence of a training dataset, it is bound to learn from its experience.

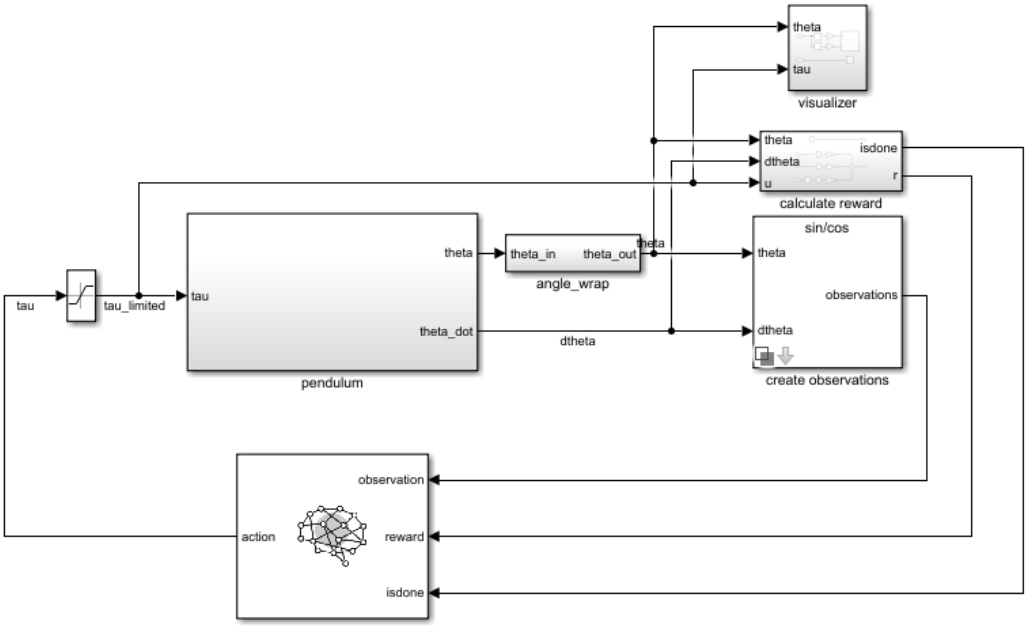
RL is the science of decision making. It is about learning the optimal behavior in an environment to obtain maximum reward. In RL, the data is accumulated from machine learning systems that use a trial-and-error method. Data is not part of the input that we would find in supervised or unsupervised machine learning.

Reinforcement learning uses algorithms that learn from outcomes and decide which action to take next. After each action, the algorithm receives feedback that helps it determine whether the choice it made was correct, neutral or incorrect. It is a good technique to use for automated systems that have to make a lot of small decisions without human guidance.

It is an autonomous, self-teaching system that essentially learns by trial and error. It performs actions with the aim of maximizing rewards, or in other words, it is learning by doing in order to achieve the best outcomes.

Main points in Reinforcement learning

- Input: The input should be an initial state from which the model will start
- Output: There are many possible outputs as there are a variety of solutions to a particular problem
- Training: The training is based upon the input,The model will return a state and the user will decide to reward or punish the model based on its output.
- The model keeps continues to learn.
- The best solution is decided based on the maximum reward.



ssFig.2. rlSimplePendulumModel

Elements of Reinforcement Learning

1. **Policy:** Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.
2. **Reward function:** Reward function is used to define a goal in a reinforcement learning problem. A reward function is a function that provides a numerical score based on the state of the environment
3. **Value function:** Value functions specify what is good in the long run. The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.
4. **Model of the environment:** Models are used for planning.
    - Credit assignment problem: Reinforcement learning algorithms learn to generate an internal value for the intermediate states as to how good they are in leading to the goal. The learning decision maker is called the agent. The agent interacts with the environment that includes everything outside the agent.
    - The agent has sensors to decide on its state in the environment and takes action that modifies its state. The reinforcement learning problem model is an agent continuously interacting with an environment. The agent and the environment interact in a sequence of time steps. At each time step t, the agent receives the state

of the environment and a scalar numerical reward for the previous action, and then the agent then selects an action.

- Reinforcement learning is a technique for solving Markov decision problems. Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards. This framework is intended to be a simple way of representing essential features of the artificial intelligence problem.

4.1 Application of Reinforcement Learnings

Robotics: Robots with pre-programmed behavior are useful in structured environments, such as the assembly line of an automobile manufacturing plant, where the task is repetitive in nature.

1. A master chess player makes a move. The choice is informed both by planning, anticipating possible replies and counter replies.
2. An adaptive controller adjusts parameters of a petroleum refinery's operation in real time.

RL can be used in large environments in the following situations:

1. A model of the environment is known, but an analytic solution is not available;
2. Only a simulation model of the environment is given (the subject of simulation-based optimization)
3. The only way to collect information about the environment is to interact with it.

4.2 Advantages of Reinforcement learning

1. Reinforcement learning can be used to solve very complex problems that cannot be solved by conventional techniques.
2. The model can correct the errors that occurred during the training process.
3. In RL, training data is obtained via the direct interaction of the agent with the environment
4. Reinforcement learning can handle environments that are non-deterministic, meaning that the outcomes of actions are not always predictable. This is useful in real-world applications where the environment may change over time or is uncertain.
5. Reinforcement learning can be used to solve a wide range of problems, including those that involve decision making, control, and optimization.
6. Reinforcement learning is a flexible approach that can be combined with other machine learning techniques, such as deep learning, to improve performance.

4.3 Disadvantages of Reinforcement learning

1. Reinforcement learning is not preferable to use for solving simple problems.
2. Reinforcement learning needs a lot of data and a lot of computation
3. Reinforcement learning is highly dependent on the quality of the reward function. If the reward function is poorly designed, the agent may not learn the desired behavior.
4. Reinforcement learning can be difficult to debug and interpret. It is not always clear why the agent is behaving in a certain way, which can make it difficult to diagnose and fix problems.

5 Proposed System

The primary objective is to train an SAC agent to control a Kinova Gen3 robot arm for a ball-balancing task**.** Imagine a scenario where the robot arm holds a ping pong ball at the center of a flat surface (plate) attached to its gripper. The challenge lies in maintaining the ball's balance despite its inherent instability.

5.1 System Modelling

We construct the physical components of the system using Simscape Multibody components. The system includes the robot arm, the ball, and the plate, integrated into a dynamic framework. The plate is constrained to the end effector of the manipulator, ensuring its movement mirrors that of the arm. With six degrees of freedom, the ball enjoys unrestricted motion in space. To model the interaction between the ball and the plate, we employ the Spatial Contact Force block, simulating contact forces. Control inputs for the manipulator are represented by torque signals applied to the actuated joints, facilitating precise manipulation. This setup lays the foundation for simulating interactions between the robot arm, the ball, and the plate within a dynamic environment. Through accurate modeling and control inputs, we can simulate realistic scenarios and study the behavior of the system under various conditions. This approach enables us to develop and refine control strategies for tasks such as ball balancing, where precise manipulation and interaction dynamics are crucial. Overall, integrating Simscape Multibody components provides a robust framework for simulating and analyzing complex mechanical systems like robotic manipulators interacting with dynamic objects.

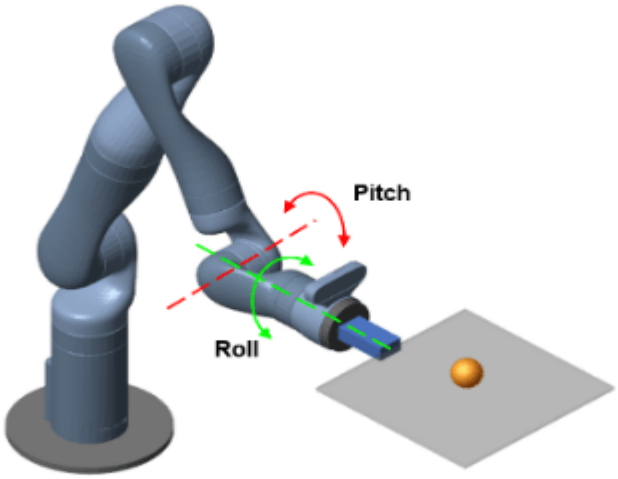


Fig.3. Kinova Gen3 robot

5.2 Observations and Actions

The SAC (Soft Actor-Critic) agent interacts with the environment using observations and actions. Observations consist of a 22-element vector comprising: sine and cosine representations of joint angles and velocities for the two actuated joints, positions (x and y distances from the plate center) and velocities of the ball, orientation (represented by quaternions) and velocities of the plate, joint torques from the previous time step, as well as the ball's radius and mass. Actions are normalized values representing joint torques. This comprehensive observation space provides the agent with essential information about the state of the environment, enabling it to make informed decisions and learn effective control strategies. By processing these observations and selecting appropriate actions, the agent can navigate the dynamic system to achieve tasks such as ball balancing. The use of normalized joint torque values as actions allows for smooth and precise control over the manipulator, facilitating efficient interaction with the environment. Overall, this observation-action setup equips the SAC agent with the necessary information and control capabilities to effectively learn and optimize its performance in the given task domain.

5.3 Simulink Model

In our Simulink model, we integrate the robot manipulator and the SAC (Soft Actor-Critic) agent within the Kinova Ball Balance subsystem, which encapsulates the robot arm and the ball-balancing task. Within this setup, the SAC agent applies actions to the robot subsystem and receives observation, reward, and termination signals in return. Notably, only the final two joints of the robot arm are actuated, primarily for pitch and roll control, focusing on the essential degrees of freedom required for the ball-balancing task. This model enables seamless

interaction between the agent and the physical system, facilitating the learning process and optimization of control strategies for achieving stable ball balancing. Through this integration, we can study and refine the agent's performance in real-time scenarios, fostering efficient and robust ball-balancing capabilities in dynamic environments.

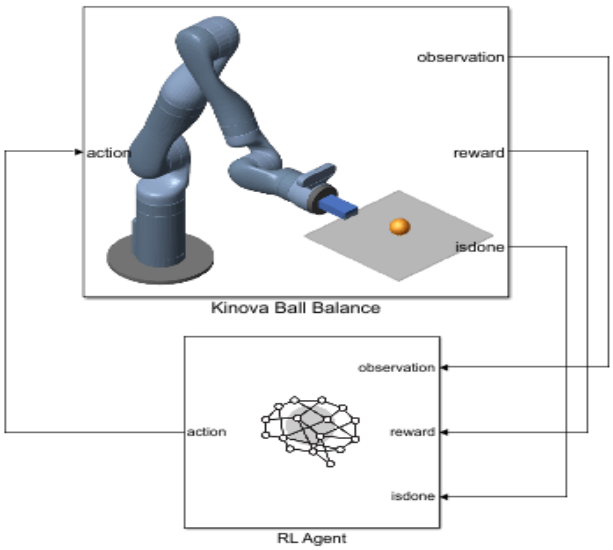


Fig.4. Kinova Ball Balance subsystem

5.4 Visualization

If the Robotics System Toolbox Robot Library Data support package is installed, users can utilize the Mechanics Explorer to access a 3D animation of the manipulator. To enable this feature, set the Visualization parameter of the 7 DOF Manipulator subsystem to 3D Mesh. However, if the support package is not installed, it's crucial to set the Visualization parameter to None. This ensures compatibility and proper functionality of the Simulink model regardless of the availability of the support package. By adhering to this setup, users can seamlessly utilize the Simulink model, ensuring it remains functional irrespective of the support package's presence. This approach provides flexibility and ensures that users can still utilize the model effectively even if certain optional features are not available. Additionally, it promotes consistency and ease of use across different environments and user configurations, enhancing the overall usability and accessibility of the Simulink model for a wider audience.

5.5 Training the SAC Agent

In our development process, we employ the Reinforcement Learning Toolbox to create and train the SAC (Soft Actor-Critic) agent. The primary goal is to train the agent to balance

the ball effectively while showcasing resilience to environmental variations. Throughout the training phase, the agent's policy and value functions undergo iterative adjustments to maximize the cumulative reward obtained from interactions with the environment. This iterative process involves fine-tuning the agent's decision-making mechanisms to facilitate the desired behavior of proficiently balancing the ball, even in the presence of uncertainties or alterations in the environment.

The SAC algorithm, renowned for its effectiveness in continuous control tasks, is utilized due to its capacity to handle continuous action spaces and uncertainty. Through the training iterations, the agent learns to optimize its control policies to achieve stable ball balancing. The reward signal, provided by the environment based on the agent's actions, serves as feedback for guiding the learning process. By maximizing cumulative rewards over multiple episodes, the agent gradually refines its policy to adapt to various environmental conditions and achieve optimal performance.

During training, the agent explores different actions and their consequences, leveraging experience to update its policy and value functions. This exploration-exploitation trade-off is crucial for striking a balance between discovering new strategies and exploiting known effective ones. Through reinforcement learning, the agent learns to navigate the complex state space efficiently, acquiring the necessary skills to balance the ball while mitigating disturbances and uncertainties.

The robustness of the trained agent is evaluated by assessing its performance across diverse environmental conditions, including variations in ball dynamics, external disturbances, and changes in system parameters. By exposing the agent to a range of scenarios during training, it learns to generalize its learned policies effectively, enabling it to adapt to unseen conditions robustly.

Overall, the training process involves a continuous refinement of the agent's control strategies, guided by reinforcement learning principles. By optimizing policy and value functions to maximize cumulative rewards, the agent becomes proficient in balancing the ball while demonstrating robustness to environmental variations, thus achieving the desired task objectives effectively.

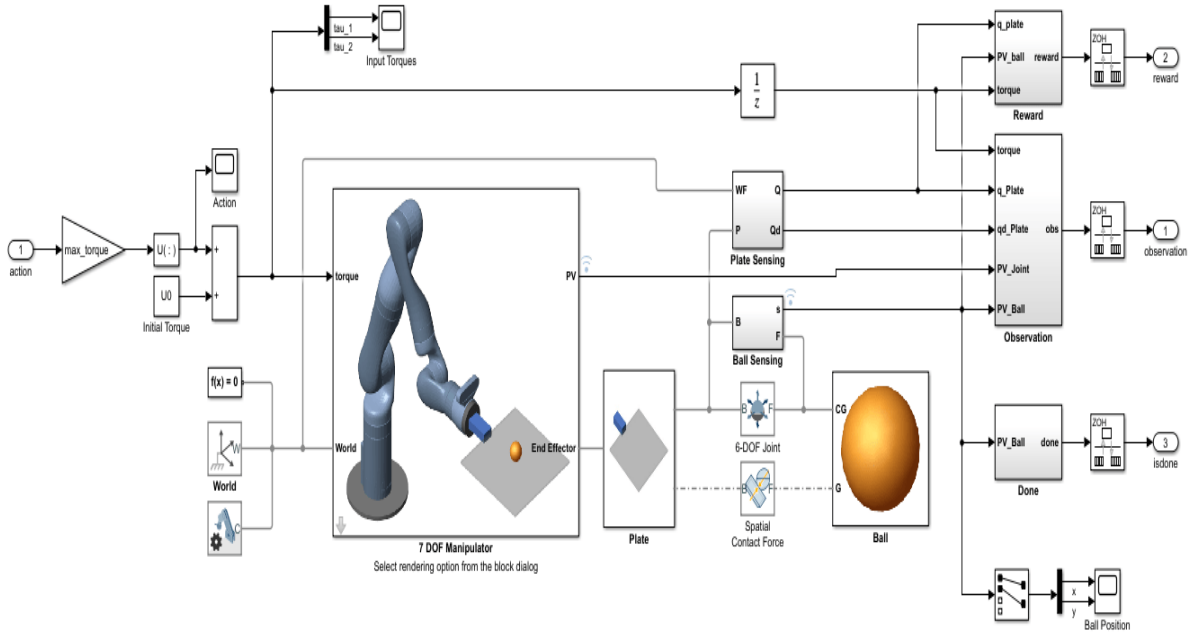


Fig.5. 7 DOF Manipulator subsystem

In this model:

- Simscape™ Multibody™ components are used to model the manipulator, ball, and plate, which are the physical components of the system.
- The plate is limited to the manipulator's end effector.
- With six degrees of freedom, the ball is able to travel about freely in space.
- The Spatial Contact Force block is used to model the contact forces between the plate and the ball.
- The torque signals for the actuated joints are the manipulator's control inputs.

5.6 Define Environment

To train a reinforcement learning agent, you must define the environment with which it will interact. For the ball balancing environment:

- The observations are represented by a 22 element vector that contains information about the positions (sine and cosine of joint angles) and velocities (joint angle derivatives) of the two actuated joints, positions (x and y distances from plate center) and velocities (x and y derivatives) of the ball, orientation (quaternions) and velocities (quaternion derivatives) of the plate, joint torques from the last time step, ball radius, and mass.The actions are normalized joint torque values.

- The sample time is Ts =0.01s, and the simulation time is Tf=10s.
- The simulation terminates when the ball falls off the plate.The sample time is Ts =0.01s, and the simulation time is Tf=10s.
- The simulation terminates when the ball falls off the plate.The reward r_t at time step t is

$$r_t = r_{\text{ball}} + r_{\text{plate}} + r_{\text{action}}$$
$$r_{\text{ball}} = e^{-0.001\left(x^2 + y^2\right)}$$
$$r_{\text{plate}} = -0.1\left(\phi^2 + \theta^2 + \psi^2\right)$$
$$r_{\text{action}} = -0.05\left(\tau_1^2 + \tau_2^2\right)$$

given by:

Here, rball is a reward for the ball moving closer to the center of the plate, rplate is a penalty for plate orientation, and raction is a penalty for control effort. ϕ, ɵ, and ψ are the respective roll, pitch, and yaw angles of the plate in radians. Ƭ1 and Ƭ2 are the joint torques.

5.7 Create Agent

The agent in this example is a soft actor-critic (SAC) agent. SAC agents use one or two parametrized Q-value function approximators to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output (the estimated discounted cumulative long-term reward for which receives the action from the state corresponding to the current observation, and following the policy thereafter). For more information on SAC agents, see Soft Actor-Critic (SAC) Agents.

The SAC agent in this example uses two critics. To model the parametrized Q-value functions within the critics, use a neural network with two input layers (one for the observation channel, as specified by obsInfo, and the other for the action channel, as specified by actInfo) and one output layer (which returns the scalar value). For more information on creating deep neural networks for reinforcement learning agents, see Create Policies and Value Functions.

Soft Actor-critic agents use a parametrized stochastic policy over a continuous action space, which is implemented by a continuous Gaussian actor. This actor takes an observation as input and returns as output a random action sampled from a Gaussian probability distribution. To approximate the mean values and standard deviations of the Gaussian distribution, you must use a neural network with two output layers, each having as many elements as the dimension of the action space. One output layer must return a vector containing the mean values for each action dimension. The other must return a vector containing the

standard deviation for each action dimension.

```
actorNet = dlnetwork;
actorNet = addLayers(actorNet,commonPath);
actorNet = addLayers(actorNet,meanPath);
actorNet = addLayers(actorNet,stdPath);
actorNet = connectLayers(actorNet,"commonPath","meanFC/in");
actorNet = connectLayers(actorNet,"commonPath","stdFC/in");
```

Since standard deviations must be nonnegative, use a softplus or ReLU layer to enforce nonnegativity. The SAC agent automatically reads the action range from the UpperLimit and LowerLimit properties of actInfo (which is used to create the actor), and then internally scales the distribution and bounds the action. Therefore, do not add a tanhLayer as the last nonlinear layer in the mean output path.

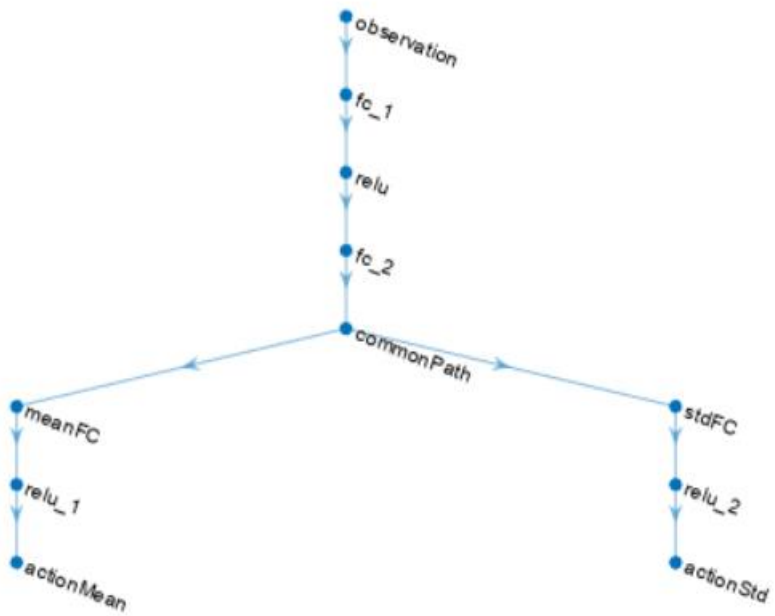


Fig.6. Actor Neural Network

5.8 Train Agent

To train the agent, first specify the training options using rlTrainingOptions. For this example, use the following options:

Run each training for at most 6000 episodes, with each episode lasting at most floor(Tf/Ts) time steps.

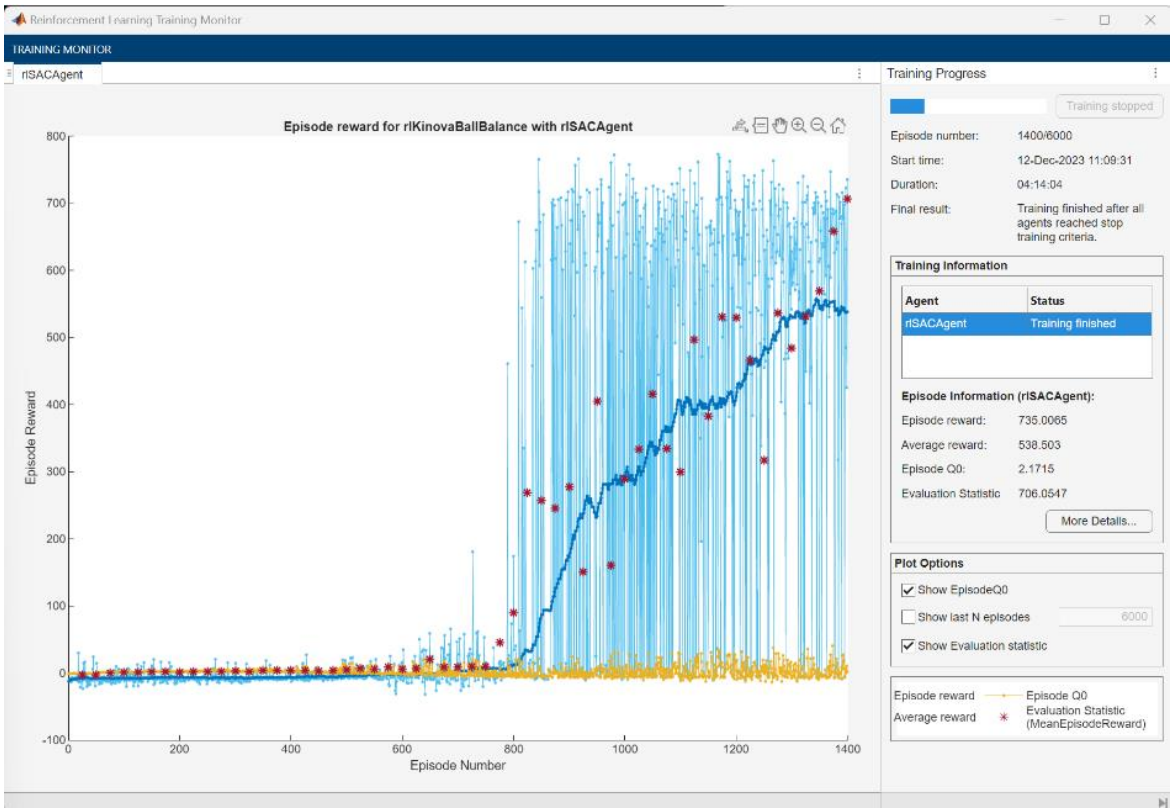


Fig.7. Episode reward for rlKinovaBallBalance with rlSACAgent

- Stop training when the evaluation of the greedy policy exceeds 700.
- To speed up training set the UseParallel option to true, which requires Parallel Computing Toolbox™ software. If you do not have the software installed set the option to false.

5.9 Simulate Trained Agent

Specify an initial position for the ball with respect to the plate center. To randomize the initial ball position during simulation, set the userSpecifiedConditions flag to false.

```
userSpecifiedConditions = true;
if userSpecifiedConditions
    ball.x0 = 0.075;
    ball.y0 = -0.075;
    env.ResetFcn = [];
else
    env.ResetFcn = @kinovaResetFcn;
end
```

Create a simulation options object for configuring the simulation. The agent will be simulated for a maximum of floor(T_f/T_s) steps per simulation episode.

```
simOpts = rlSimulationOptions(MaxSteps=floor(Tf/Ts));
```

Enable visualization during simulation.

```
set_param(mdl, SimMechanicsOpenEditorOnUpdate="on");
doViz = true;
```

Simulate the agent using the greedy policy.

```
agent.UseExplorationPolicy = false;
experiences = sim(agent,env,simOpts);
```

To train a Soft Actor-Critic (SAC) agent for ball balancing control, you first need to set up the environment, potentially creating a custom one if it doesn't exist in OpenAI Gym. Ensure you have the necessary dependencies installed, such as TensorFlow and Gym. Define the SAC agent using TensorFlow, including an actor and two critics. The actor network generates mean actions and log standard deviations, while the critics estimate the Q-values. Implement a training loop to train the agent in the environment, updating its parameters based on experiences. Make sure to adjust hyperparameters and architecture as needed. After training, save the model weights for later use. While this outlines a basic approach, it's essential to consider the complexity and nuances of implementing SAC effectively. Utilizing existing libraries like TensorFlow Agents or Stable Baselines can streamline this process.

View the trajectory of the ball using the Ball Position scope block.

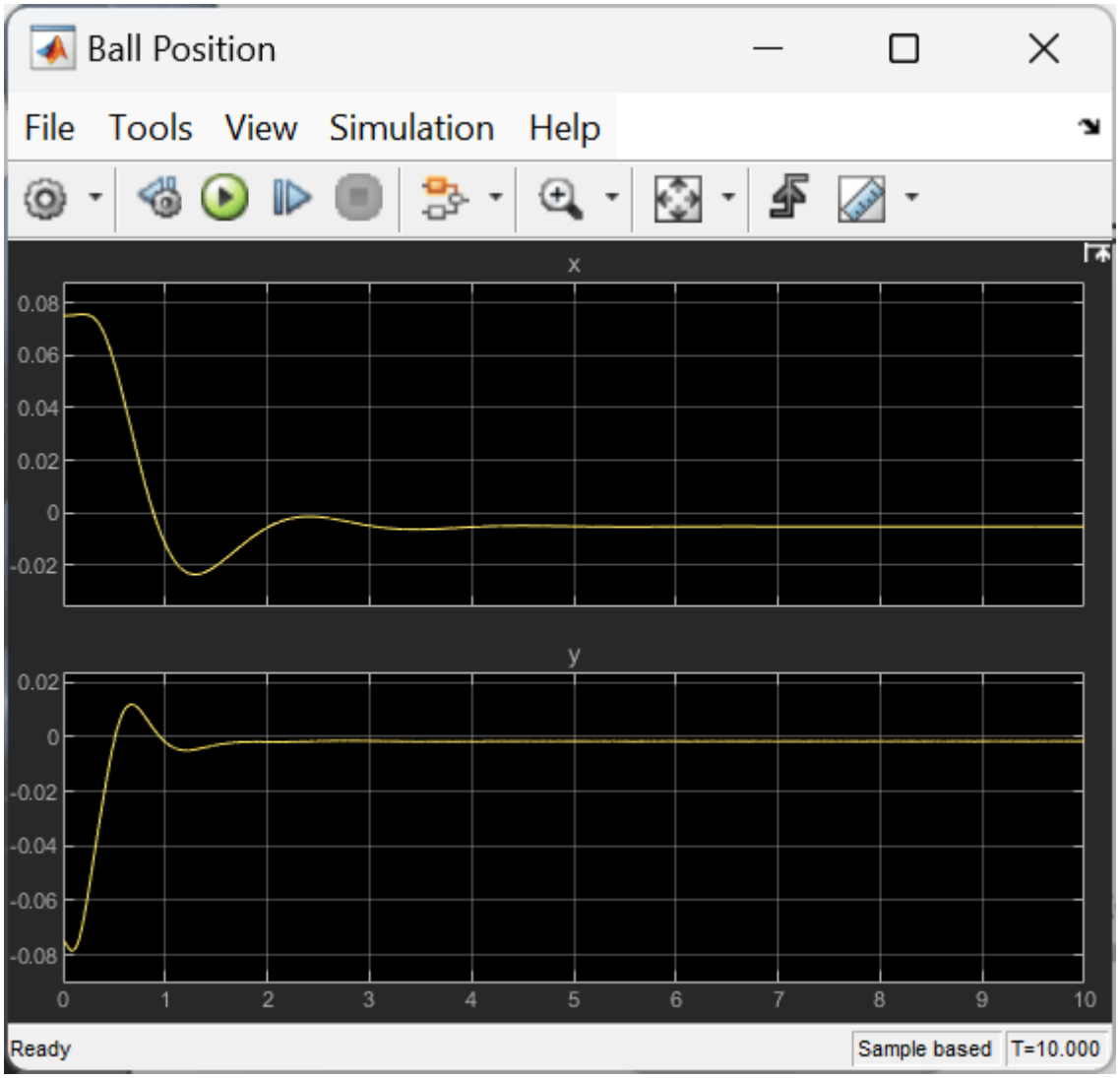


Fig.8. Ball Position

6 CONCLUTION

In this paper, RL algorithm is superposed to the conventional controller, making up a compound controller. Both the conventional feedback controller and the compound controller can stabilize the ballbot system, while the compound controller has larger recovery area. When the compound controller tries to balance the ballbot from an initial state that the conventional controller fails, the ballbot “kicks” the ball, which is an unmodelable but interesting phenomenon. This control policy can only be designed by RL. It is demonstrated that under certain circumstances, RL-based approach could give better or even unpredictable policies that can rarely be derived by conventional control framework.

7 REFERENCES

[1] H. Bang and Y. S. Lee, "Implementation of a Ball and Plate Control System Using Sliding Mode Control," in IEEE Access, vol. 6, pp. 32401-32408, 2018, doi: 10.1109/ACCESS.2018.2838544.

[2] F. Pan, D. Xue, D. Chen and Jianjiang Cui, "Design and implementation of rotary inverted pendulum motion control hardware-in-the-loop simulation platform," 2010 Chinese Control and Decision Conference, Xuzhou, 2010, pp. 2328- 2333, doi: 10.1109/CCDC.2010.5498818.

[3] Huang Mei and Zhang He, "Study on stability control for single link rotary inverted pendulum," 2010 International Conference on Mechanic Automation and Control Engineering, Wuhan, 2010, pp. 6127-6130, doi: 10.1109/MACE.2010.5536653.

[4] N. S. A. Aziz, R. Adnan and M. Tajjudin, "Design and evaluation of fuzzy PID controller for ball and beam system," 2017 IEEE 8th Control and System Graduate Research Colloquium (ICSGRC), Shah Alam, 2017, pp. 28-32, doi: 10.1109/ICSGRC.2017.8070562.

[5] T. Anjali and S. S. Mathew, "Implementation of optimal control for ball and beam system," 2016 International Conference on Emerging Technological Trends (ICETT), Kollam, 2016, pp. 1-5, doi: 10.1109/ICETT.2016.7873763.

[6] Dong Zhe, Zheng Geng and Liu Guoping, "Networked nonlinear model predictive control of the ball and beam system," 2008 27th Chinese Control Conference, Kunming, 2008, pp. 469-473, doi: 10.1109/CHICC.2008.4605795.

[7] M. M. Kopichev, A. A. Kuznetsov, A. R. Muzalevskiy and T. L. Rusyaeva, "Ball and Beam Stabilization Laboratory Test Bench With Intellectual Control," 2020 XXIII International Conference on Soft Computing and Measurements (SCM), St. Petersburg, Russia, 2020, pp. 112-116, doi: 10.1109/SCM50615.2020.9198776.

[8] M. T. Ho, Y. Rizal, and L. M. Chu, “Visual servoing tracking control of a ball and plate system: Design, implementation and experimental validation,” Int. J. Adv. Robot. Syst., vol. 10, 2013, doi: 10.5772/56525.

[9] E. Fabregas, S. Dormido-Canto, and S. Dormido, “Virtual and Remote Laboratory with the Ball and Plate System,” IFACPapersOnLine, vol. 50, no. 1, pp. 9132–9137, 2017, doi: 10.1016/j.ifacol.2017.08.1716.

[10] X. Dong, Z. Zhang, and C. Chen, “Applying genetic algorithm to on-line updated PID neural network controllers for ball and plate system,” 2009 4th Int. Conf. Innov. Comput. Inf. Control. ICICIC 2009, pp. 751–755, 2009, doi: 10.1109/ICICIC.2009.113.

[11] F. I. R. Betancourt, S. M. B. Alarcon, and L. F. A. Velasquez, “Fuzzy and PID controllers applied to ball and plate system,” 4th IEEE Colomb. Conf. Autom. Control Autom. Control as Key Support Ind. Product. CCAC 2019 - Proc., pp. 1–6, 2019, doi: 10.1109/CCAC.2019.8921113.

[12] A. Adiprasetya and A. S. Wibowo, “Implementation of PID controller and pre-filter to control non-linear ball and plate system,” ICCEREC 2016 - Int. Conf. Control. Electron. Renew. Energy, Commun. 2016, Conf. Proc., pp. 174–178, 2017, doi: 10.1109/ICCEREC.2016.7814965.

[13] I. M. Mehedi, U. M. Al-Saggaf, R. Mansouri, and M. Bettayeb, “Two degrees of freedom fractional controller design: Application to the ball and beam system,” Meas. J. Int. Meas. Confed., vol. 135, pp. 13–22, 2019, doi: 10.1016/j.measurement.2018.11.021.